

ArtesanosEnLinea

PROYECTO PARA EL CFGS DE DESARROLLO DE
APLICACIONES WEB. CURSO: 2025/2026

Alumno: Celestino Fernando Amigo de Pascual

INDICE

1. Introducción.....	5
1.1. Breve idea-resumen de la idea del proyecto.....	5
1.2. Objeto/ Finalidad del proyecto.....	5
2. Metodología y Contenidos.....	6
2.1. Planificación para la tarea.....	6
2.2. Sincronización con github (sistema de control de versiones).....	7
3. Arquitectura del sistema.....	8
3.1. Estructura inicial del MVC.....	9
3.2. Diseño Frontend (Interfaz y Experiencia de Usuario).....	10
3.2.1. Mapa de Vistas e Interfaz de Usuario.....	11
3.2.1.1. Vista 1: Página de Inicio (Landing Page).....	11
3.2.1.2. Vista 2: El Catálogo de Productos.....	12
3.2.1.3. Vista 3: Carrito y Formulario de Pago.....	13
3.2.1.4. Vista 4: Historial de "Mis Pedidos".....	13
3.2.1.5. Vista 5: Panel del Administrador.....	14
3.2.1.6. Vista 6: Pantallas de Acceso y Registro.....	14
3.3. Diseño Backend (La lógica que hace de motor).....	16
3.3.1. Controladores.....	16
3.3.1.1. Principales métodos de los controladores.....	17
3.3.2. Modelos.....	18
3.3.2.1. Principales métodos en los modelos.....	19
3.3.3. Las vistas.....	20
3.4. La estructura y la creación de la BBDD.....	22
3.4.1. Diccionario de Datos y Tablas Principales.....	23
3.4.1.1. Tabla 1: usuarios (Almacena los perfiles de acceso).....	23
3.4.1.2. Tabla 2: categorias (Clasificación del catálogo).....	23
3.4.1.3. Tabla 3: productos (El inventario de la tienda).....	23
3.4.1.4. Tabla 4: pedidos (Cabecera de las facturas).....	24

3.4.1.5.	Tabla 5: detalles_pedido (Líneas individuales de compra)	24
3.4.2.	Integridad Referencial (Reglas de borrado)	25
3.4.3.	Usuarios de prueba y Seguridad	25
4.	Pruebas y Validación (Testing)	27
4.1.	Pruebas de Seguridad y Control de Acceso	27
4.1.1.	Caso de Prueba: Seguridad. Protección de rutas de Administrador.	27
4.1.2.	Caso de Prueba: Seguridad (Cifrado de contraseñas)	28
4.2.	Pruebas de Interfaz y Validación (Formularios)	28
4.2.1.	Caso de Prueba: (Validación de Registro en JavaScript)	28
4.2.2.	Caso de Prueba: Campos obligatorios en el Pago	29
4.3.	Pruebas de Lógica de Negocio e Integridad (Backend)	29
4.3.1.	Caso de Prueba: Reducción automática de stock	29
4.3.2.	Caso de Prueba: Integridad de la Transacción SQL (Rollback)	30
4.3.3.	Caso de Prueba: Persistencia del carrito de compra	30
5.	Despliegue de la aplicación	31
5.1.	Despliegue local en XAMPP	31
5.2.	Despliegue en Producción (Entorno Real con Docker y DDNS) (Anexo, instalación en Docker)	33
5.2.1.	Resumen de accesos y puntos de entrada	34
6.	Resultados y discusión	36
7.	Conclusiones	37
8.	Anexos	38
8.1.	Glosario	38
8.2.	Referencias bibliográficas	38
8.3.	Guía de instalación y Manual de Usuario	39
8.3.1.	Guía de instalación	39
8.3.2.	Manual de usuario. Casos de uso	39
8.3.2.1.	El cliente finaliza una compra	39
8.3.2.2.	El administrador añade una nueva artesanía	40

8.3.2.3.	Cambio de estado de un envío	40
8.4.	Extractos de código reseñables.....	41

1. Introducción.

1.1. Breve idea-resumen de la idea del proyecto.

El presente proyecto consiste en el diseño, desarrollo e implantación de una plataforma de comercio electrónico denominada ArtesanosEnLinea, especializada en la comercialización de productos artesanales grabados en madera. La idea surge de que estoy en contacto con artesanos en mi entorno, y me parece buena idea proporcionarles una herramienta digital propia que permita gestionar sus ventas sin depender de plataformas de terceros, ofreciendo una experiencia de usuario personalizada y adaptada a la naturaleza del producto. Trabajan esencialmente la madera.

1.2. Objeto/ Finalidad del proyecto

El proyecto trata de demostrar cómo se puede construir una web totalmente funcional mediante el uso de técnicas y tecnologías utilizadas durante el curso, sin la necesidad de utilización de frameworks o el uso de plantillas predefinidas (tipo wordpress).

2. Metodología y Contenidos

El proyecto será llevado con trabajo escalonado en varios días y poco a poco, recogiendo los avances en un repositorio de github abierto al público, para que pueda ser examinado por el tribunal.

En cuanto a las tecnologías escogidas (al menos inicialmente, sin perjuicio que según el tiempo disponible pueda llevar a cabo ampliaciones), son las siguientes.

- Para la estructura de la web, uso de lenguaje HTML5.
- Para los estilos, uso de CSS.
- Para la interactividad en lado cliente, Javascript.
- Para la base de datos, MySQL.
- Para el desarrollo del back-end, PHP.

2.1. Planificación para la tarea

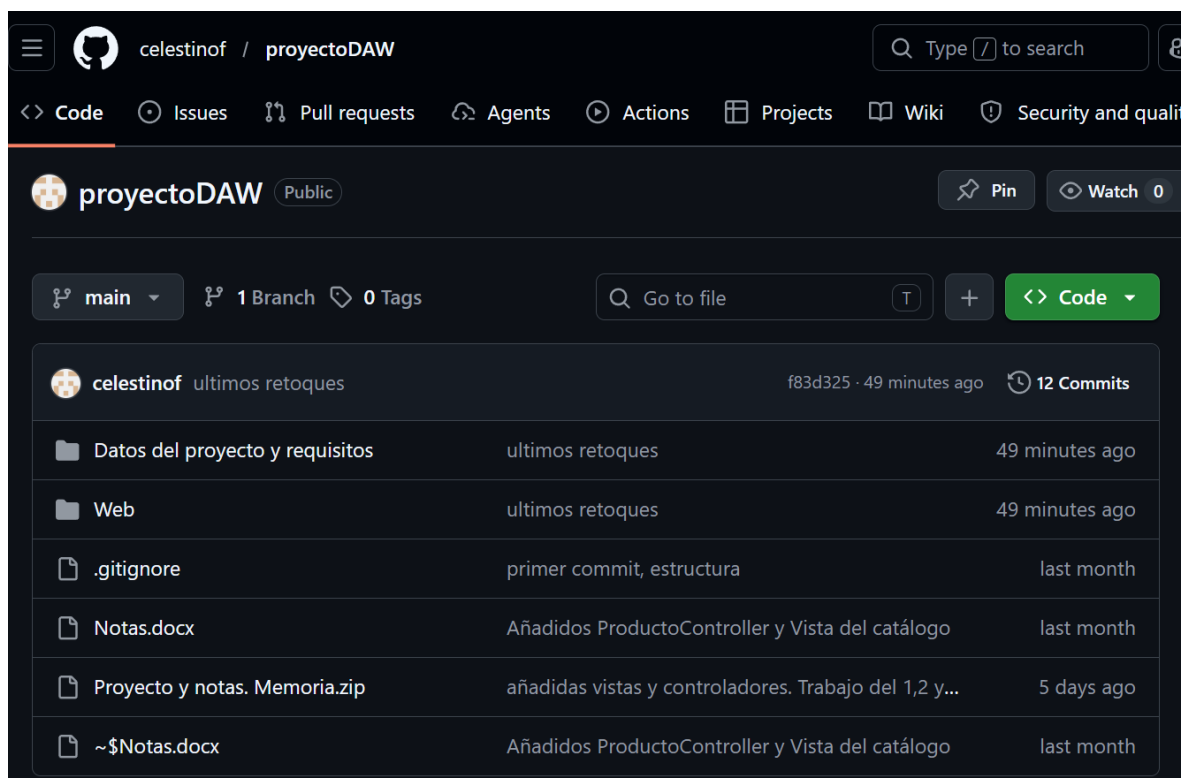
Voy a tratar de organizarme de la siguiente manera, aunque como se sabe muchas veces los planes solo son guías que varían con el tiempo, creo que una forma coherente de planificar el trabajo del proyecto es la siguiente:

1. Primera parte. Análisis de los requisitos del proyecto, diseño de BBDD y estructura del Modelo Vista Controlador. Aproximadamente 4 días / 1 semana
2. Segunda parte. Desarrollo de CRUD (lo visto en el módulo de php, siguiendo un esquema de los videos proporcionados en el curso (“garaje codepills”). 1 semana / 10 días.
3. Tercera parte. Implementar el carrito y los pedidos. Si es tiempo añadir pasarela de pago (1 semana/10 días).
4. Cuarta y última parte. Maquetación final. CSS, Bootstrap, pruebas. 1 semana aproximadamente.

2.2. Sincronización con github (sistema de control de versiones).

De manera breve explico la sincronización llevada a cabo con github, así como indico el acceso al repositorio público.

1. Inicio git init en mi pc para crear repositorio git.
2. Creo .gitignore para evitar que se sincronice todo el composer.
3. Vincular repositorio remoto a github: `git remote add origin https://github.com/celestinof/proyectoDAW.git`
4. Renombrar la rama a “main”, la nueva barra de github.
5. Subir los archivos a la nube: `git push -u origin main`



3. Arquitectura del sistema

El sistema está estructurado bajo el patrón de diseño MVC (Modelo-Vista-Controlador), organizando el código según su responsabilidad:

- **El Controlador Frontal (public/index.php):** Es el punto de acceso único de la aplicación. Cualquier petición que haga el usuario a través del navegador pasa primero por este archivo. Su función principal es analizar la dirección web (URL), identificar qué acción está solicitando el usuario y delegar la tarea al Controlador correspondiente. Además, actúa como primera barrera de seguridad, comprobando el estado de la sesión antes de permitir el acceso a áreas restringidas (administradores).
- **El Controlador (src/Controllers/...):** Es el coordinador lógico del proceso. Recibe las instrucciones del Controlador Frontal y organiza los pasos a seguir. Su labor es capturar la información, validarla, pasársela al Modelo para que la procese, y decidir qué pantalla (Vista) debe mostrarse.
- **El Modelo (src/Models/...):** Es el gestor de los datos. Es la única capa del sistema que se comunica con la base de datos (MySQL). Mediante conexiones seguras, ejecuta las consultas necesarias para guardar, actualizar, eliminar o extraer información sobre productos, usuarios o pedidos registrados.
- **La Vista (src/views/...):** Es la interfaz visual. Son los archivos de diseño estructurados con HTML y Bootstrap. Su misión es exclusiva de presentación: reciben la información procesada por el Controlador y la muestran en pantalla. Se adaptan dinámicamente según el contexto (por ejemplo, mostrando botones de gestión de catálogo únicamente si el usuario identificado es un administrador).

PRIMERA PARTE.

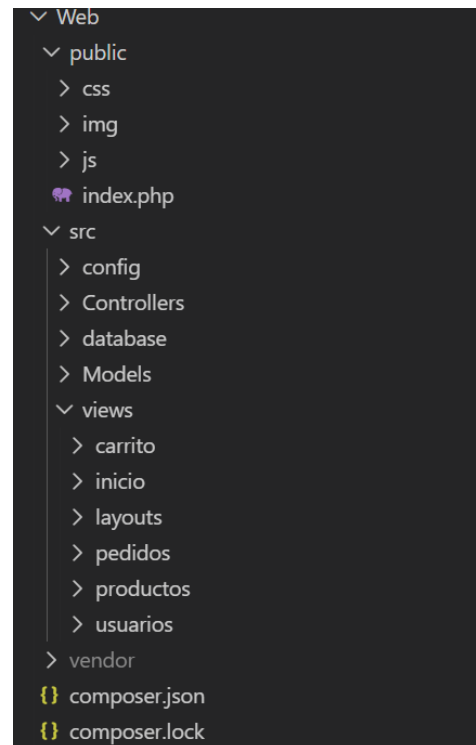
3.1. Estructura inicial del MVC

En la carpeta Datos del proyecto y requisitos guardo la memoria a presentar, así como otros datos de interés.

En base a los apuntes de clase, he dado el siguiente formato a la organización de la web en base al visto en asignaturas como DWES.

Por otro lado, hemos utilizado **composer** para que gestione directamente el autoload y evitar estar utilizando require en cada una de las páginas.

Breve descripción de cada una de las carpetas:



- **public:** El único directorio accesible desde la web, y quizás por ello el más importante.
 - Index.php (el front controller).
 - Las carpetas **/css**, **/js**, **/img** para guardar dicho contenido.
- Dentro de **src**:
 - **config:** Archivos de configuración. Por ejemplo un archivo conexión.php para la conexión con la BBDD usando PDO (Lo aprendido en DWES).
 - **controllers:** Lógica de la aplicación. Las páginas que contienen la lógica para la interacción del usuario.
 - **models:** Interacción con la base de datos (como un almacén). Se creará una clase PHP por cada tabla en la BBDD.
 - **views:** Plantillas de visualización (HTML/PHP). Pintar los datos que le envía el controlador. Aquí es donde se aplican estilos CSS.

3.2. Diseño Frontend (Interfaz y Experiencia de Usuario)

Para construir la parte visual de **ArtesanosEnLinea**, he preferido hacer todo el diseño desde cero en lugar de utilizar plantillas prefabricadas (como las de WordPress). He utilizado HTML y CSS normales, apoyándome en una herramienta gratuita llamada **Bootstrap**. Lo he cargado vía Content Delivery Network (CDN), no localmente (es decir, sin haber descargado los archivos) para asegurar que la aplicación sea *Responsive* (adaptable a dispositivos variados). Elegí CDN por que según leí en foros tipo stackoverflow (por ejemplo, en este hilo <https://stackoverflow.com/questions/26192897/should-i-use-bootstrap-from-cdn-or-make-a-copy-on-my-server>), en el que se comenta que el CDN da una carga rápida, e incluso que puede ser que esos archivos ya estén en caché al haber visitado otras webs. De todas maneras, aunque no tuve tiempo, no estaría de más tener una copia en local que descargue en defecto del CDN.

Los principales componentes empleados y su justificación son:

- **Navbar (Barra de navegación):** Utilizada en el header.php para gestionar el menú principal, los accesos condicionales según el rol del usuario (Cliente/Admin) y el acceso al carrito. ([Documentación de Bootstrap Navbar](#))
- **Cards (Tarjetas):** Empleadas en la vista principal del catálogo (productos/index.php) para encapsular de forma elegante la imagen, el título, la descripción y el precio de cada artesanía. ([Documentación de Bootstrap Cards](#))
- **Modals (Ventanas emergentes):** Implementadas para mostrar información sin obligar al usuario a cambiar de página, mejorando la retención. Se utilizan para el "Contacto" global y para el aviso de "Cancelación de pedido" en el panel del cliente. ([Documentación de Bootstrap Modal](#))
- **Badges (Insignias):** Utilizadas intensivamente en las tablas de pedidos (mis_pedidos.php y gestion_admin.php) para identificar visualmente, mediante un código de colores (Verde, Amarillo, Rojo, Azul), el estado actual del envío. ([Documentación de Bootstrap Badges](#))
- **Collapse/Accordion (Desplegables):** Integrado en el historial de compras del cliente para ocultar o mostrar dinámicamente los detalles (productos concretos) de un

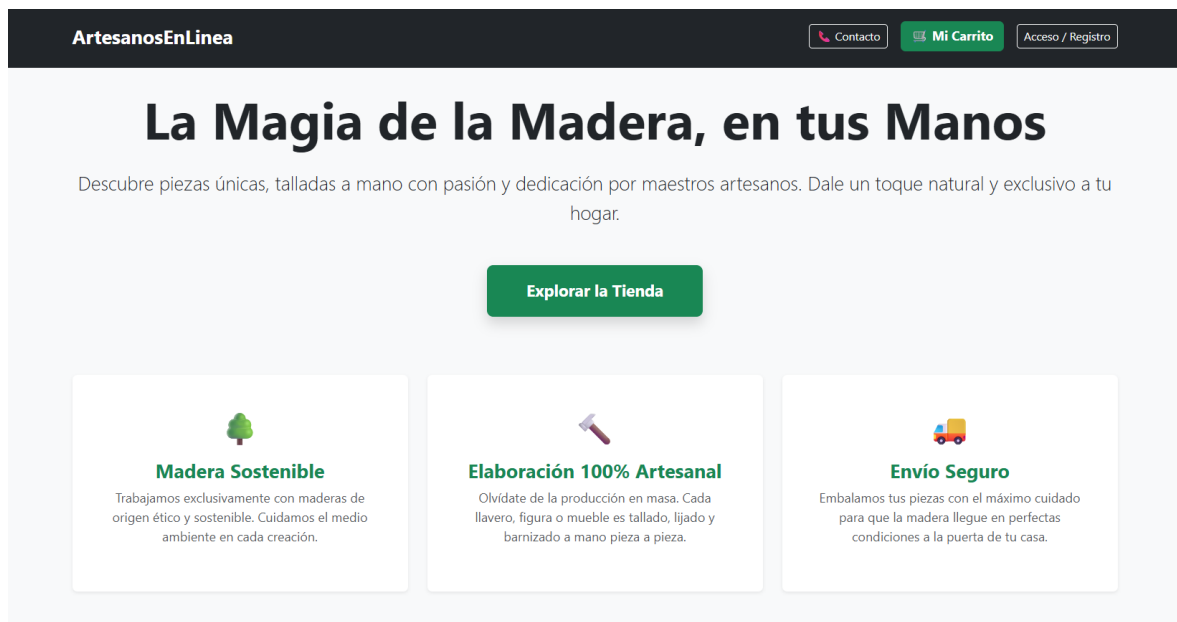
pedido general al pulsar un botón, manteniendo la tabla limpia. ([Documentación de Bootstrap Collapse](#))

Además, he usado **JavaScript** para comprobar que los formularios estén bien rellenos antes de enviarlos (por ejemplo, que el cliente no deje su contraseña en blanco). Esto hace que la página sea más rápida y dé menos errores.

A continuación, explico qué hace cada pantalla de la web y cómo está diseñada para ser fácil de usar:

3.2.1. Mapa de Vistas e Interfaz de Usuario

3.2.1.1. Vista 1: Página de Inicio (Landing Page)



- **Ruta del archivo:** `src/views/inicio/index.php`
- **Descripción:** Es la puerta de entrada a la tienda. Su objetivo es dar la bienvenida y explicar rápidamente que se venden artesanías de madera.
- **Diseño:** Tiene el menú de navegación arriba y una imagen grande de presentación con un mensaje que invita al visitante a entrar a ver el catálogo.

3.2.1.2. Vista 2: El Catálogo de Productos

ArtesanosEnLinea

ContactoMi CarritoAcceso / Registro

Catálogo de Productos

Todos los artículosRegalos PersonalizadosRegalos Estándar



Llavero Personalizado Madera
Llavero de roble con nombre grabado a doble cara

13.00 € Stock: 44

Personalización (Opcional):
Ej: Grabar la fecha '12/05/2026'

Te contactaremos con un boceto antes de fabricarlo.

Añadir al Carrito



Llavero Acero Inoxidable
Llavero resistente con forma de corazón y fecha grabada

15.00 € Stock: 28

Personalización (Opcional):
Ej: Grabar la fecha '12/05/2026'

Te contactaremos con un boceto antes de fabricarlo.

Añadir al Carrito



Llavero de Cuero
Llavero de cuero trenzado con iniciales

9.90 € Stock: 43

Personalización (Opcional):
Ej: Grabar la fecha '12/05/2026'

Te contactaremos con un boceto antes de fabricarlo.

Añadir al Carrito

- **Ruta del archivo:** src/views/productos/index.php
- **Descripción:** Es el escaparate de la tienda donde se muestran todos los artículos disponibles.
- **Diseño:** Los productos se muestran ordenados en una cuadrícula. Cada artículo tiene su propio recuadro blanco con su foto, el nombre, el precio y un botón verde muy visible para añadirlo al carrito. Además, se muestra el stock, y en caso de ser un artículo personalizable, un recuadro en el que rellenar la información.
- **A tener en cuenta:** Si la persona que entra es el administrador de la tienda, en esos mismos recuadros aparecen unos botones extra para poder modificar el precio o borrar el producto sin tener que irse a otra página.

3.2.1.3. Vista 3: Carrito y Formulario de Pago

Producto	Precio Ud.	Cantidad	Subtotal	Quitar
Llavero Personalizado Madera	15,73 €	1	15,73 €	
TOTAL PEDIDO:			15,73 €	

Seguir Comprando

Vaciar Carrito Finalizar Compra

© 2026 ArtesanosEnLinea - Proyecto Final DAW

- **Rutas de los archivos:** src/views/carrito/ver.php y src/views/carrito/pago.php
- **Descripción:** Son las pantallas donde el cliente revisa lo que va a comprar y pone sus datos para que se lo enviemos.
- **Diseño:** Primero se muestra una tabla muy clara con la lista de los artículos elegidos y el precio total con el IVA ya calculado. Si el cliente está conforme, pasa a un formulario muy sencillo donde solo tiene que escribir la calle y ciudad a la que quiere que mandemos el paquete. Si pincha en “Seguir comprando” vuelve al catálogo para seguir añadiendo ítems.

3.2.1.4. Vista 4: Historial de "Mis Pedidos"

Nº Pedido	Fecha de Compra	Dirección de Envío	Total (IVA inc.)	Estado del Envío
#00013 Ver artículos	08/06/2026 10:07	calle aventura 15	15,73 €	PENDIENTE Solicitar Cancelación
#00013 Ver artículos	08/06/2026 10:07	calle aventura 15	15,73 €	PENDIENTE Solicitar Cancelación

© 2026 ArtesanosEnLinea - Proyecto Final DAW

- **Ruta del archivo:** src/views/pedidos/mis_pedidos.php
- **Descripción:** Es la zona privada del cliente para ver las compras que ha hecho en el pasado.

- **Diseño:** Consiste en una tabla resumen. Utiliza etiquetas de colores llamativos para que el cliente sepa rápido cómo va su paquete (por ejemplo, verde si está "Enviado" o amarillo si está "Pendiente"). Además, si el cliente hace clic en un pedido, la fila se abre hacia abajo enseñando qué artículos exactos iban dentro de esa compra, ahorrando así mucho espacio en la pantalla.
- **Para cancelar pedidos:** Debido a que se trata en teoría de una actividad con baja demanda y servicio personalizado, se opta por la vía de contacto para la cancelación por parte del gestor de la tienda.

3.2.1.5. Vista 5: Panel del Administrador

Nº Pedido	Datos del Cliente	Fecha	Total	Gestión de Estado
#00013	Carlos García carlos.garcia@gmail.com	08/06/2026	15,73 €	Pendiente de pago Guardar
#00012	Carlos García carlos.garcia@gmail.com	08/06/2026	30,13 €	Pendiente de pago Guardar

- **Ruta del archivo:** src/views/pedidos/gestion_admin.php
- **Descripción:** Es la pantalla desde donde el dueño de la tienda controla todas las ventas.
- **Diseño:** Muestra una tabla con todos los pedidos de la web, indicando quién lo ha comprado y a qué dirección hay que mandarlo. Lo más útil es que incluye unos menús desplegables para que el administrador pueda cambiar el estado de un paquete (por ejemplo, marcarlo como "Enviado") directamente desde ahí con un solo clic.

3.2.1.6. Vista 6: Pantallas de Acceso y Registro

Acceso

ArtesanosEnLinea [Contacto](#) [Mi Carrito](#) [Acceso / Registro](#)

Identificación

Correo Electrónico:

Contraseña:

[Entrar al Sistema](#)

¿No tienes cuenta para comprar?

[Regístrate aquí](#)

[Volver al catálogo](#)

Registro

ArtesanosEnLinea [Contacto](#) [Mi Carrito](#) [Acceso / Registro](#)

Crear Cuenta de Cliente

Nombre Completo:

Correo Electrónico:

Contraseña:

[Registrarme](#)

[Ya tengo cuenta](#)

© 2026 ArtesanosEnLinea - Proyecto Final DAW

- **Rutas de los archivos:** `src/views/usuarios/login.php` y `src/views/usuarios/registro.php`
- **Descripción:** Son las pantallas para que los usuarios nuevos se creen una cuenta o para que los que ya tienen una inicien sesión.
- **Diseño:** Son formularios de acceso o de registro. Inicialmente sale el de acceso, pero pulsando en el botón de registro se pasa al registro.

3.3. Diseño Backend (La lógica que hace de motor).

3.3.1. Controladores

En lugar de tener todo el código mezclado en un solo archivo, y por la costumbre de la POO que hemos tenido durante el curso, he modularizado la lógica de la web en "Controladores" (Clásico del MVC). Cada controlador se encarga de una tarea específica.

- **Controlador de Usuarios:** La seguridad de la sesión. Se encarga de comprobar que los correos y las contraseñas sean correctos al iniciar sesión. Por seguridad, las contraseñas nunca se guardan normales en la base de datos, sino que se transforman en un código secreto (encriptación) para que ni siquiera el administrador pueda leerlas. Todas las contraseñas son 123456, por lo que en la BBDD desde phpMyAdmin veremos el mismo hash en todas.
- **Controlador de Productos:** Transporte de los datos (se lo pide al modelo y lo lleva a la vista). Se encarga de ir a la base de datos, recoger la información de las artesanías y enviarla a la pantalla para que el cliente las vea.

Si entra el administrador, este controlador le da permiso para subir fotos nuevas, cambiar precios o borrar artículos (es decir, permisos adicionales).

- **Controlador del Carrito:** Es la cesta de la compra temporal. Para no saturar la base de datos con carritos que la gente abandona, este controlador guarda los artículos elegidos en la memoria temporal del navegador en las famosas variables Super Globales que utilizamos durante todo el curso en DWEC, la `$_SESSION`.
- **Controlador de Pedidos:** Hace varias funciones a la vez, y quizás es la parte más delicada. Cuando un cliente paga, este controlador hace tres cosas al mismo tiempo: guarda el pedido, apunta la dirección de envío y resta la madera o el artículo del inventario. Si algo falla a la mitad (por ejemplo, se cae internet), cancela toda la operación automáticamente para que no haya errores de cobro.

3.3.1.1. Principales métodos de los controladores

Archivo PHP	Métodos programados	¿Qué hace exactamente este código?
InicioController.php	index()	Simplemente carga la página de bienvenida y el menú principal cuando alguien entra a la web.
ProductoController.php	- index() - crear()/guardar() -editar()/actualizar() - eliminar()	La función index() extrae el catálogo de la base de datos para mostrárselo al cliente. El resto de funciones (crear, guardar, editar, eliminar) conforman el "CRUD" exclusivo para el administrador, incluyendo el código que coge la foto subida y la guarda en la carpeta del servidor.
UsuarioController.php	-login()/procesarLogin() -registro()/guardarUsuario() - logout()	Las funciones de registro y login recogen los datos del formulario. Utilizan una herramienta interna de PHP llamada password_hash() para encriptar la contraseña. La función logout() se encarga de destruir la sesión abierta por seguridad cuando el usuario se marcha.
CarritoController.php	- ver() - agregar()	Estas funciones trabajan directamente sobre la memoria temporal del navegador (\$_SESSION['carrito']). Permiten meter

	- eliminar() - procesar()	artículos, borrarlos si el cliente se arrepiente, y atrapar la dirección de envío justo antes de mandar la orden de compra.
PedidoControlle r.php	- misPedidos() - gestionAdmin() - actualizarEstado()	misPedidos() busca el historial de un solo cliente. gestionAdmin() busca todos los pedidos de la web. actualizarEstado() es la función que permite cambiar la palabra "Pendiente" por "Enviado" en la base de datos.

3.3.2. Modelos

En la arquitectura MVC, por pura seguridad, los Controladores tienen prohibido hablar directamente con la base de datos. Como una analogía de los módulos del ciclo (no sé si fue en DWEC o DWES), si el "Controlador" es el dependiente de la tienda, el "Modelo" es el trabajador del almacén: es el único que tiene la llave para entrar, buscar en las estanterías (tablas de MySQL) y devolverle la información al dependiente.

He programado tres modelos principales, todos ellos conectados a la base de datos mediante **PDO**, que es la tecnología que utilizamos durante el curso y que utiliza "sentencias preparadas" (un sistema de seguridad que impide que los hackers puedan inyectar código malicioso a través de los formularios).

- **Modelo de Productos (Producto.php):** Gestiona la conexión directa con la base de datos para todo lo relacionado con el catálogo de la tienda. Se encarga de ejecutar las consultas necesarias para extraer la lista de artesanías disponibles y enviarlas a la pantalla. Además, procesa las peticiones del administrador cuando este necesita insertar un nuevo artículo de madera, actualizar su precio y stock, o retirarlo definitivamente del catálogo.

- **Modelo de Usuarios (Usuario.php):** Controla el acceso a la información de los clientes y administradores. Se encarga de insertar de forma segura los datos de un nuevo usuario en la base de datos durante el proceso de registro. Durante el inicio de sesión, este modelo realiza la consulta para comprobar si el correo electrónico introducido existe y extrae la contraseña encriptada, devolviendo esta información al controlador para que valide si el acceso es correcto.
- **Modelo de Pedidos (Pedido.php):** Gestiona el guardado de las compras y la actualización del inventario. Es un archivo crítico porque agrupa varias acciones simultáneas. Cuando un cliente finaliza una compra, este modelo interactúa con la base de datos ejecutando tres operaciones a la vez: registra los datos generales de la factura, guarda el detalle exacto de los artículos adquiridos, y resta automáticamente las unidades compradas del stock general. Si alguna de estas tres acciones falla por algún error, el modelo deshace toda la operación para garantizar que los datos nunca se descuadren.

3.3.2.1. Principales métodos en los modelos

Archivo PHP	Métodos programados	¿Qué hace exactamente en la Base de Datos?
Producto.php	- listarTodo() - obtenerPorId() - insertar() - actualizar() - eliminar()	Ejecuta consultas SELECT para pintar el catálogo y devolver los datos de una artesanía. Ejecuta las consultas INSERT, UPDATE y DELETE cuando el administrador añade, modifica o descataloga productos del inventario.
Usuario.php	- registrar() - obtenerPorEmail()	El método registrar() ejecuta un INSERT seguro de los datos del cliente. El método

		obtenerPorEmail() hace un SELECT buscando por correo electrónico para recuperar las credenciales durante el proceso de login.
Pedido.php	- procesarCheckout() - obtenerPedidosPorUsuario() - obtenerTodosLosPedidos() - obtenerDetallesPorPedido() - actualizarEstado()	procesarCheckout() utiliza Transacciones SQL (beginTransaction, commit, rollBack) para insertar la factura, los detalles y restar el stock como un bloque irrompible. Los demás métodos usan SELECT para extraer historiales y un UPDATE para que el administrador pueda cambiar el estado del paquete.

3.3.3. Las vistas

Para completar el patrón de arquitectura MVC, es necesario definir el papel de las **Vistas**. Mientras que los Modelos extraen los datos y los Controladores deciden qué hacer con ellos, las Vistas son los archivos encargados de recibir esa información y fusionarla con el código HTML para mostrársela al usuario.

Por seguridad y buenas prácticas, las Vistas tienen prohibido ejecutar consultas a la base de datos o contener lógica de negocio compleja. Su único trabajo es utilizar pequeñas etiquetas de PHP (como bucles foreach o instrucciones echo) para imprimir en pantalla las variables que el Controlador les ha enviado previamente.

Para estructurar las vistas de manera eficiente y evitar la repetición masiva de código, el directorio src/views/ se ha organizado con la siguiente lógica:

1. El sistema de Layouts (Plantillas Maestras) En lugar de escribir la etiqueta <html>, el menú de navegación y las importaciones de Bootstrap en todas y cada una de las páginas, se ha creado una subcarpeta llamada layouts que contiene dos archivos vitales:

- **header.php:** Contiene la cabecera del documento, las llamadas a los estilos CSS y la barra de navegación superior. Este archivo incluye lógica condicional para detectar si hay una sesión iniciada y cambiar dinámicamente los botones (por ejemplo, ocultar "Iniciar sesión" y mostrar "Cerrar sesión" o el panel de "Admin").
- **footer.php:** Cierra las etiquetas principales del HTML e incluye las llamadas a los archivos de JavaScript y las ventanas modales globales.

2. Inyección de Vistas Dinámicas Gracias a este sistema de *Layouts*, el resto de las vistas de la aplicación (como el catálogo, el carrito o los formularios) solo contienen el código estricto de su sección central. Cuando el Controlador necesita cargar una pantalla, simplemente une las tres piezas como si fuera un sándwich utilizando la instrucción `require_once`: primero carga el header, luego la vista solicitada y finalmente el footer.

3. Organización por Módulos Para mantener el proyecto ordenado y escalable, las vistas centrales se han separado en carpetas según el área a la que pertenecen:

- **/inicio:** Contiene la portada (`index.php`).
- **/productos:** Vistas del catálogo y los formularios de creación/edición del administrador.
- **/carrito:** La tabla de resumen (`ver.php`) y el formulario de envío (`pago.php`).
- **/pedidos:** Los paneles de gestión del cliente (`mis_pedidos.php`) y del administrador (`gestion_admin.php`).
- **/usuarios:** Las pantallas de autenticación (`login.php` y `registro.php`).

3.4. La estructura y la creación de la BBDD

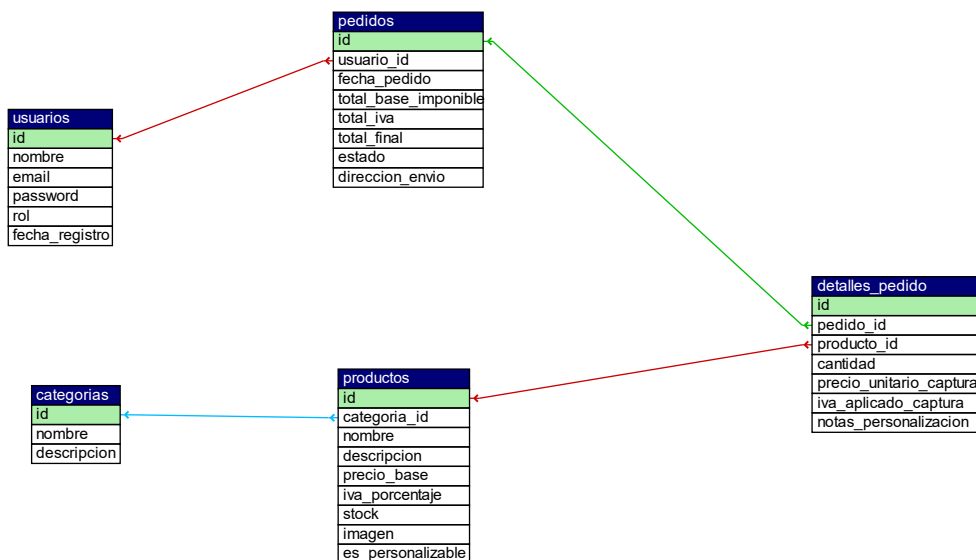
Esta parte se trata de un punto crítico, puesto que necesitamos la base de datos para conectar usuarios con artículos y pedidos. En base a lo estudiado en el módulo de bases de datos, la idea es crear una Base de datos **relacional**, con las siguientes tablas.

He dejado el archivo .sql de la base de datos con las indicaciones de las tablas y atributos para que pueda ser revisada fácilmente. Se encuentra en **/src/database/artesanos_db.sql**

Para importarla, yo lo hago desde PHPMyAdmin, tal como hicimos en el curso. Adjunto una captura de la BBDD.



Vista de la Base de datos en modo diseño:



Introduce algunos valores para verificar el funcionamiento de los modelos. Generamos un script SQL con sentencias INSERT guardando el esquema de las tablas.

3.4.1. Diccionario de Datos y Tablas Principales

Para comprender la naturaleza de los datos almacenados, se detalla a continuación la estructura exacta de las cinco tablas que componen el sistema, extraídas directamente del diccionario de datos de phpMyAdmin, indicando el tipo de dato de cada columna, sus valores predeterminados y su propósito:

3.4.1.1. **Tabla 1: usuarios** (Almacena los perfiles de acceso)

Columna	Tipo de Dato	Nulo	Descripción / Restricción
id	int(11)	No	Clave Primaria.
nombre	varchar(100)	No	Nombre completo del usuario.
email	varchar(150)	No	Correo electrónico. Tiene índice UNIQUE (no puede haber dos iguales).
password	varchar(255)	No	Contraseña encriptada por el sistema.
rol	enum('admin', 'cliente')	Sí	Define los permisos. Predeterminado: 'cliente'.
fecha_registro	timestamp	No	Se rellena automáticamente con current_timestamp().

3.4.1.2. **Tabla 2: categorias** (Clasificación del catálogo)

Columna	Tipo de Dato	Nulo	Descripción / Restricción
id	int(11)	No	Clave Primaria.
nombre	varchar(100)	No	Nombre de la categoría (ej. Llaveros, Grabados).
descripcion	text	Sí	Explicación opcional. Predeterminado: NULL.

3.4.1.3. **Tabla 3: productos** (El inventario de la tienda)

Columna	Tipo de Dato	Nulo	Descripción / Restricción
id	int(11)	No	Clave Primaria.
categoria_id	int(11)	Sí	Clave Foránea. Predeterminado: NULL.

nombre	varchar(150)	No	Título descriptivo de la artesanía.
descripcion	text	Sí	Texto detallado. Predeterminado: NULL.
precio_base	decimal(10,2)	No	Precio sin impuestos.
iva_porcentaje	decimal(5,2)	Sí	Porcentaje de impuesto. Predeterminado: 21.00.
stock	int(11)	Sí	Cantidad de unidades físicas. Predeterminado: 0.
imagen	varchar(255)	Sí	Nombre del archivo de la foto. Predeterminado: NULL.
es_personalizable	tinyint(1)	Sí	Booleano para saber si admite grabado. Predeterminado: 1.

3.4.1.4. **Tabla 4: pedidos** (Cabecera de las facturas)

Columna	Tipo de Dato	Nulo	Descripción / Restricción
id	int(11)	No	Clave Primaria.
usuario_id	int(11)	Sí	Clave Foránea (BTREE). Predeterminado: NULL.
fecha_pedido	timestamp	No	Se captura automáticamente con current_timestamp().
total_base_imponible	decimal(10,2)	Sí	Suma sin IVA. Predeterminado: NULL.
total_iva	decimal(10,2)	Sí	Impuestos. Predeterminado: NULL.
total_final	decimal(10,2)	Sí	Coste total. Predeterminado: NULL.
estado	enum	Sí	Admite: 'pendiente', 'pagado', 'enviado', 'cancelado'. Predeterminado: 'pendiente'.
direccion_envio	varchar(255)	No	Domicilio introducido en el formulario de pago.

3.4.1.5. **Tabla 5: detalles_pedido** (Líneas individuales de compra)

Columna	Tipo de Dato	Nulo	Descripción / Restricción
id	int(11)	No	Clave Primaria.
pedido_id	int(11)	Sí	Clave Foránea (BTREE). Predeterminado: NULL.
producto_id	int(11)	Sí	Clave Foránea (BTREE). Predeterminado: NULL.

cantidad	int(11)	No	Número de unidades adquiridas.
precio_unitario_captura	decimal(10,2)	No	Precio congelado en el momento del pago.
iva_aplicado_captura	decimal(5,2)	No	IVA congelado en el momento del pago.
notas_personalizacion	text	Sí	Instrucciones del cliente. Predeterminado: NULL.

3.4.2. Integridad Referencial (Reglas de borrado)

Un aspecto fundamental del diseño ha sido la configuración de las Claves Foráneas para evitar que la base de datos se corrompa si se eliminan registros. Se han aplicado dos reglas principales:

- **Borrado en Cascada (ON DELETE CASCADE):** Si se elimina un usuario de la tabla usuarios, el sistema borrará automáticamente todos sus pedidos de la tabla pedidos y sus detalles para no dejar datos huérfanos.
- **Borrado Nulo (ON DELETE SET NULL):** Si el administrador decide eliminar un producto del catálogo, este desaparecerá de la tienda, pero no afectará a los historiales de compra pasados. La base de datos mantendrá la línea del pedido intacta, simplemente marcando el producto_id como NULL, garantizando así la coherencia contable.

3.4.3. Usuarios de prueba y Seguridad

Para verificar el correcto funcionamiento de los modelos de conexión, he generado un script SQL con sentencias INSERT que incluye algunos datos predefinidos.

Usuarios de pruebas:

Para las pruebas con la BBDD establecí algunos usuarios (un administrador y cuatro clientes). La contraseña para todos ellos es **123456**.

nombre	email
Admin Tienda	admin@artesanos.com
Carlos García	carlos.garcia@gmail.com
Laura Martínez	laura.martinez@hotmail.com
Miguel Ángel	miguel.angel@yahoo.com
LARA	lara@gmail.com

Sin embargo, a nivel de seguridad, es importante destacar que **esta contraseña no está guardada en texto plano en la base de datos**. Al revisar la tabla usuarios en phpMyAdmin, el campo password mostrará una cadena de caracteres en formato hash. (De hecho si se abre la BBDD, se puede ver que todos tienen el mismo hash, al tener la misma contraseña).

4. Pruebas y Validación (Testing)

Para garantizar la robustez, seguridad y correcta funcionalidad de **ArtesanosEnLinea**, se ha llevado a cabo una fase de pruebas funcionales exhaustivas antes del despliegue final. En lugar de automatizar las pruebas, se ha optado por realizar pruebas manuales de "Caja Negra", simulando el comportamiento tanto de un usuario inexperto (que comete errores sin querer) como de un usuario malintencionado (que intenta saltarse la seguridad).

Las pruebas se han dividido en tres categorías principales:

1. **Pruebas de Interfaz (Frontend):** Comprobar que los formularios avisan de los errores antes de enviar nada al servidor.
2. **Pruebas de Seguridad (Acceso):** Comprobar que nadie puede entrar donde no debe.
3. **Pruebas de Lógica de Negocio (Backend y BBDD):** Comprobar que el dinero y el stock se calculan perfectamente.

A continuación, se detallan los Casos de Prueba más críticos ejecutados durante la validación del proyecto:

4.1. Pruebas de Seguridad y Control de Acceso

4.1.1. Caso de Prueba: Seguridad. Protección de rutas de Administrador.

Campo	Detalle
Descripción	Verificar que un cliente normal o un visitante anónimo no pueda ver el listado global de pedidos de la tienda.
Acción realizada	Iniciar sesión con una cuenta de rol cliente. Escribir manualmente en la barra de direcciones del navegador la ruta directa del panel del jefe: <code>index.php?controller=Pedido&action=gestionAdmin</code>
Resultado esperado	El sistema debe detectar que el rol no es admin, bloquear la carga de la página y redirigir al usuario al <code>index.html</code> . En mi caso, te devuelve a <code>public/index.php</code>

Resultado obtenido	El <i>Front Controller</i> intercepta la petición, verifica la variable <code>\$_SESSION['rol']</code> , deniega el acceso y redirige correctamente a la portada.
Estado	ÉXITO

4.1.2. Caso de Prueba: Seguridad (Cifrado de contraseñas)

Campo	Detalle
Descripción	Comprobar que el sistema de encriptación de PHP funciona en la base de datos.
Acción realizada	Registrar un nuevo usuario desde la web con la contraseña 123456. Abrir phpMyAdmin y revisar la tabla usuarios.
Resultado esperado	El campo password no debe mostrar "123456", sino un texto ilegible (Hash).
Resultado obtenido	El campo muestra el hash generado por el cifrado de contraseñas.
Estado	ÉXITO

4.2. Pruebas de Interfaz y Validación (Formularios)

4.2.1. Caso de Prueba: (Validación de Registro en JavaScript)

Campo	Detalle
Descripción	Evitar que un usuario se registre si se equivoca al repetir su contraseña.
Acción realizada	En el formulario de registro, rellenar el campo "Contraseña" con hola123 y el campo "Repetir Contraseña" con adios123. Pulsar "Registrarse".
Resultado esperado	El formulario no debe enviarse al servidor. Debe aparecer un aviso en rojo indicando que las contraseñas no coinciden.
Resultado obtenido	El script de JavaScript manipula el DOM, frena el envío (haciendo un <code>preventDefault()</code>) y muestra la alerta visual en la pantalla.
Estado	ÉXITO

4.2.2. Caso de Prueba: Campos obligatorios en el Pago

Campo	Detalle
Descripción	Evitar que un paquete se envíe sin dirección.
Acción realizada	Llegar al paso final del carrito y dejar el campo de direccion_envio completamente en blanco. Pulsar "Confirmar Pedido".
Resultado esperado	El formulario no debe enviarse. El navegador debe advertir al usuario de que la dirección es obligatoria para poder hacer el envío.
Resultado obtenido	El atributo HTML5 required del campo bloquea la petición al servidor y el navegador muestra una alerta nativa indicando: "Completa este campo".
Estado	ÉXITO

4.3. Pruebas de Lógica de Negocio e Integridad (Backend)

4.3.1. Caso de Prueba: Reducción automática de stock

Campo	Detalle
Descripción	Comprobar que el inventario se actualiza de forma autónoma cada vez que un cliente realiza una compra válida.
Acción realizada	Revisar el stock del "Llavero Personalizado Madera" en el panel (Ej: 44 unidades). Añadir 3 unidades al carrito, rellenar la dirección y finalizar el pago. Volver a revisar el catálogo.
Resultado esperado	El stock del llavero en la base de datos debe haber bajado exactamente a 41 unidades tras la compra.
Resultado obtenido	La sentencia UPDATE programada dentro de la Transacción del Modelo de Pedidos resta la cantidad exacta adquirida por el cliente.
Estado	ÉXITO

4.3.2. Caso de Prueba: Integridad de la Transacción SQL (Rollback)

Muy importante. Especialmente en el caso de pagos.

Campo	Detalle
Descripción	Asegurar que no se guarden pedidos corruptos si ocurre un fallo a mitad del guardado (por ejemplo, si falla la conexión a la base de datos al restar el stock).
Acción realizada	Forzar un error en el código de Pedido.php (ej. cambiar el nombre de una tabla temporalmente) justo antes de que se ejecute la resta de stock y simular una compra.
Resultado esperado	Como la compra no se puede completar al 100%, la factura general tampoco debe guardarse para no cobrarle al cliente algo que no se le va a enviar.
Resultado obtenido	El bloque try/catch detecta el fallo. La instrucción rollBack() deshace la inserción previa del pedido, manteniendo la base de datos intacta y sin datos a medias.
Estado	ÉXITO

4.3.3. Caso de Prueba: Persistencia del carrito de compra

Campo	Detalle
Descripción	Asegurar que el cliente no pierda los artículos seleccionados si decide seguir navegando por la tienda antes de pagar.
Acción realizada	Añadir un "Retrato Grabado Láser" al carrito. Hacer clic en el logotipo para volver a la página de inicio, visitar otra categoría y finalmente pulsar el icono del carrito.
Resultado esperado	El producto añadido inicialmente debe seguir en la lista de la compra con su precio y cantidad correctos.
Resultado obtenido	La variable súper global \$_SESSION['carrito'] mantiene la información viva y disponible durante toda la navegación del usuario de forma exitosa.
Estado	ÉXITO

5. Despliegue de la aplicación

5.1. Despliegue local en XAMPP

Despliegue de la aplicación (Manual de Instalación)

Para garantizar que el tribunal o cualquier otro desarrollador pueda evaluar y ejecutar el proyecto **ArtesanosEnLinea** sin problemas de compatibilidad, se ha preparado la aplicación para ser desplegada en un entorno de desarrollo local como los que usamos durante el curso.

El sistema ha sido probado y optimizado para funcionar sobre el paquete de software **XAMPP**, el cual emula un servidor web completo proporcionando Apache, PHP y MySQL/MariaDB.

A continuación, se detalla la guía paso a paso para la correcta instalación y ejecución del proyecto desde cero:

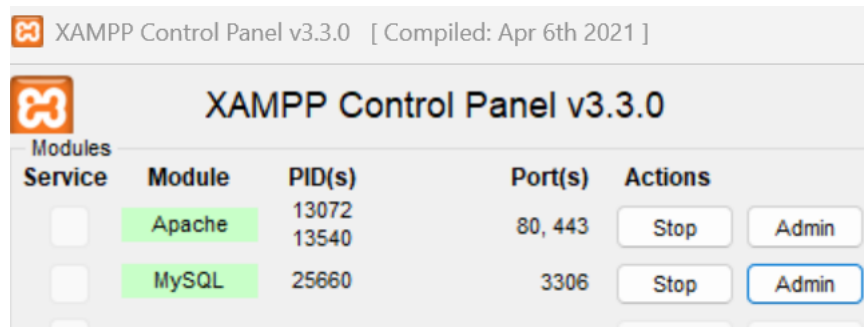
Requisitos previos del sistema

- **Servidor Local:** Tener instalado XAMPP
- **Gestor de dependencias (Opcional):** Composer instalado en el equipo (aunque la carpeta vendor con el Autoloader ya se incluye precompilada en la entrega para facilitar la evaluación).
- **Navegador Web:** Google Chrome, Mozilla Firefox o Microsoft Edge o cualquier navegador actualizado.

Guía de Instalación Paso a Paso

Paso 1: Preparación del servidor Apache y MySQL

1. Abrir el Panel de Control de XAMPP (XAMPP Control Panel).
2. Iniciar los módulos de **Apache** (Servidor Web) y **MySQL** (Base de Datos) pulsando el botón *Start* en ambos.
3. Comprobar que los indicadores de estado se iluminan en color verde, confirmando que los puertos están abiertos y a la escucha.



Paso 2: Ubicación de los archivos del proyecto

1. Descomprimir el archivo entregado con el código fuente del proyecto.
2. Copiar la carpeta principal del proyecto (recomendable renombrarla simplemente a artesanos para facilitar la navegación).
3. Pegar dicha carpeta dentro del directorio público de XAMPP. En un sistema Windows estándar, la ruta exacta sería: C:\xampp\htdocs\artesanos

Paso 3: Creación e importación de la Base de Datos

1. Abrir el navegador web e introducir la dirección: <http://localhost/phpmyadmin/>
2. En el menú lateral izquierdo, hacer clic en **Nueva** para crear una base de datos.
3. Nombrar a la base de datos estrictamente como artesanos_db.
4. Hacer clic sobre la base de datos recién creada y navegar a la pestaña superior **Importar**.
5. Pulsar en "Seleccionar archivo" y buscar el script SQL proporcionado en el código fuente, ubicado en: C:\xampp\htdocs\artesanos\src\database\artesanos_db.sql
6. Bajar hasta el final de la página y pulsar el botón **Importar**. Si el proceso es correcto, aparecerá un mensaje de éxito en color verde confirmando la creación de las 5 tablas y la inserción de los usuarios de prueba.

Paso 4: Ejecución de la aplicación Dado que la arquitectura del proyecto se basa en el patrón MVC con un *Front Controller* por motivos de seguridad, el usuario no debe acceder a la raíz de la carpeta, sino al directorio público.

1. Abrir el navegador web.

2. Escribir la siguiente URL en la barra de direcciones:
`http://localhost/artesanos/public/`
3. El sistema cargará automáticamente el archivo `index.php`, arrancará las sesiones, verificará la conexión a la base de datos y mostrará la *Landing Page* (página de bienvenida) de la tienda.

Credenciales de acceso para la evaluación

Para poder probar todas las funcionalidades del sistema (como las transacciones de compra o el panel del administrador) sin necesidad de registrar nuevos usuarios desde cero, se han habilitado las siguientes credenciales en la base de datos importada:

- **Acceso Administrador (Acceso total al CRUD y gestión de pedidos):**
 - **Email:** `admin@artesanos.com`
 - **Contraseña:** 123456
- **Acceso Cliente (Para simular compras e historial):**
 - **Email:** `lara@gmail.com` *(o cualquier otro cliente de los generados en el script. Yo por ejemplo usaba el de Carlos García. Lara lo cree desde la propia aplicación)*
 - **Contraseña:** 123456

5.2. Despliegue en Producción (Entorno Real con Docker y DDNS) (Anexo, instalación en Docker).

Para ir un paso más allá de la simple simulación local en XAMPP y demostrar los conocimientos adquiridos en administración de sistemas, la aplicación **ArtesanosEnLinea** se ha puesto en producción de forma accesible a través de Internet.

Para ello, se ha utilizado un servidor físico doméstico (un equipo *Lenovo ThinkCentre M710Q Tiny*) configurado con sistema operativo **Ubuntu Server**, ubicado en Madrid. El entorno de ejecución se administra a través de **Portainer**, una potente plataforma gráfica

de gestión de contenedores, lo que garantiza aislamiento, facilidad de mantenimiento y seguridad.

A continuación, se detalla el proceso de puesta en producción:

Paso 1: Despliegue del "Stack" a través de Portainer En lugar de instalar Apache y PHP directamente en el sistema operativo del servidor, se ha utilizado el panel web de Portainer para desplegar un nuevo *Stack* (pila de servicios). Esta pila levanta de forma simultánea e interconectada:

1. **El servicio Web:** Un contenedor configurado con Apache y PHP (versión 8.2), al cual se le han inyectado los archivos del proyecto web mediante la configuración de volúmenes persistentes.
2. **El servicio de Base de Datos:** Un contenedor ejecutando el motor MariaDB, también con almacenamiento persistente para garantizar que no se pierda el inventario ni los pedidos si el servidor se reinicia.
3. **Gestión de Base de Datos:** Contenedor PhpMyAdmin para la administración visual de las tablas relacionales.

5.2.1. Resumen de accesos y puntos de entrada

Dado que se trata de un servidor privado que tenía corriendo distintos servicios, habilité una DNS dinámica (ddns) de mi router, y dejé abiertos solo para mostrar la interoperabilidad a través de internet mediante mi dirección para redireccionar a mi ip pública. <http://celes-madrid.tplinkdns.com>

Servicio	URL de Acceso	Puerto (Interno/Externo)	Descripción
Tienda Online	http://celes-madrid.tplinkdns.com:8085/	8085 -> 80	Acceso directo a la portada del proyecto (Frontend y catálogo).

BBDD (PhpMyAdmin)	http://celes- madrid.tplinkdns.com:8086/	8086 -> 80	Panel de administración de datos para revisar tablas y transacciones
----------------------	---	------------	--

Una vez concluido el periodo de evaluación, la infraestructura se desmantelará eliminando el *Stack* desde Portainer y borrando la carpeta /home/happy/proyecto_temporal_artesanos/, asegurando así la completa limpieza de recursos en el servidor.

6. Resultados y discusión

En este apartado, presento una valoración técnica del sistema desarrollado:

- **Cumplimiento de objetivos:** La aplicación *ArtesanosEnLinea* cumple satisfactoriamente con los requisitos funcionales definidos al inicio. El sistema permite una gestión integral de productos (CRUD), un proceso de compra asistido para el usuario final y un panel de administración capaz de gestionar el inventario y el estado de los pedidos de manera centralizada.
- **Evaluación del rendimiento:** La implementación del patrón **MVC (Model-View-Controller)** ha demostrado ser una elección acertada. La separación de responsabilidades facilita el mantenimiento y la escalabilidad del código. Asimismo, el despliegue en un entorno contenerizado (**Docker/Portainer**) ha eliminado las incompatibilidades propias del desarrollo local (XAMPP), ofreciendo tiempos de carga estables y una gestión eficiente de los recursos del servidor (Lenovo M710Q).
- **Desafíos técnicos y aprendizaje:** El mayor reto ha sido la construcción poco a poco, que ha ido demostrando las carencias del proyecto sobre el análisis inicial. La falta de un apartado de gestión de pedidos, la gestión de esos pedidos por ambas partes (admin y cliente), como ejemplos a señalar.

7. Conclusiones

Este proyecto me ha permitido consolidar los conocimientos adquiridos durante el ciclo de DAW, no solo en la programación web, sino en el ciclo de vida completo de un software: desde el diseño de la base de datos relacional hasta la puesta en producción real.

- **Logros alcanzados:** He logrado desplegar una plataforma robusta, segura (mediante el cifrado BCRYPT y la protección de rutas mediante sesiones) y accesible desde Internet, utilizando tecnologías actuales como Docker.
- **Valoración personal:** Ha sido una experiencia altamente gratificante ver cómo la teoría sobre el patrón MVC se materializa en una arquitectura limpia y funcional. La capacidad de administrar un servidor real y resolver problemas de despliegue ha sido el mayor valor añadido de este trabajo.
- **Líneas de mejora futura:** Como siguiente fase, me planteo:
 1. **Pasarela de pago:** Integrar APIs reales (Stripe) para gestionar pagos online de forma segura.
 2. **Optimización:** Implementar una mejor gestión de pedidos por parte del administrador, con una nueva tabla en la BBDD que permita el acceso a pedidos eliminados y borrados del listado (esto se me ha ocurrido a punto de entregar el proyecto).
 3. **Seguridad adicional:** Configurar certificados SSL automatizados (Let's Encrypt) a través del Nginx Reverse Proxy para asegurar todo el tráfico cifrado de extremo a extremo. (Esto lo descubrí mediante el uso de algunos contenedores de uso personal).

8. Anexos

8.1. Glosario

- **Apache:** Servidor web de código abierto que procesa las peticiones HTTP y sirve el contenido.
- **Composer:** Gestor de dependencias para PHP que facilita la carga de librerías externas.
- **Docker:** Plataforma que permite empaquetar aplicaciones en "contenedores", garantizando que funcionen igual en cualquier entorno.
- **DDNS (Dynamic DNS):** Servicio que permite asignar un nombre de dominio fijo a una dirección IP dinámica.
- **MVC (Modelo-Vista-Controlador):** Patrón de arquitectura de software que separa la lógica de negocio, la interfaz de usuario y el acceso a datos.
- **PDO (PHP Data Objects):** Extensión de PHP que proporciona una interfaz uniforme para acceder a bases de datos.
- **Portainer:** Interfaz gráfica de gestión para administrar contenedores Docker de forma sencilla.
- **Reverse Proxy:** Servidor que actúa como intermediario para las peticiones de los clientes, mejorando la seguridad y gestión del tráfico.
- **Stack:** Conjunto de servicios o contenedores definidos para funcionar de manera conjunta en Docker.

8.2. Referencias bibliográficas

- **Material didáctico:**
 - Módulos de Desarrollo Web en Entorno Cliente (DWEK) y Desarrollo Web en Entorno Servidor (DWES).

- **Documentación técnica oficial:**

- *PHP Documentation:* <https://www.php.net/docs.php>
- *MariaDB Knowledge Base:* <https://mariadb.com/kb/>
- *Docker Docs:* <https://docs.docker.com/>

- **Comunidad y resolución de problemas:**

- *Stack Overflow:* Consultas sobre arquitectura MVC y manejo de PDO en entornos Linux. <https://stackoverflow.com/>
- *Reddit (r/selfhosted, r/docker):* Buenas prácticas para la gestión de contenedores y seguridad en despliegues caseros. [enlace sospechoso eliminado]
- *Docker Hub:* Documentación de imágenes oficiales (php-apache, mariadb). <https://hub.docker.com/>

8.3. Guía de instalación y Manual de Usuario

8.3.1. Guía de instalación

Esta parte ya fue tratada en la parte de “despliegue de instalación”.

8.3.2. Manual de usuario. Casos de uso

Para entender cómo funciona la web, he documentado los procesos principales que ocurren por detrás de la pantalla:

8.3.2.1. *El cliente finaliza una compra*

Paso a paso	Descripción del proceso interno
¿Quién lo hace?	Un cliente que ha iniciado sesión.
Requisito previo	Tener al menos un artículo artesanal en el carrito.
1. Inicio	El cliente entra al carrito y pulsa "Tramitar Pedido".
2. Datos	La web le pide la dirección. Al escribirla y aceptar, la información viaja oculta al Controlador de Pedidos.

3. Comprobación	El sistema calcula el total y el IVA basándose en los precios actuales.
4. Guardado seguro	El sistema comienza una "Transacción". Guarda el pedido, asocia los artículos comprados a ese pedido y resta la cantidad del stock disponible en la tienda.
5. Finalización	Se vacía el carrito temporal del cliente y se le envía a la pantalla de "Mis Pedidos" para que vea su recibo.

8.3.2.2. *El administrador añade una nueva artesanía*

Paso a paso	Descripción del proceso interno
¿Quién lo hace?	Únicamente el dueño de la tienda (Administrador).
Requisito previo	Haber iniciado sesión con una cuenta que tenga el rol de "admin".
1. Inicio	El administrador pulsa el botón "Crear Producto" en el catálogo.
2. Datos	Rellena el nombre, la descripción, el precio, el stock y selecciona una foto de su ordenador.
3. Subida de imagen	El código PHP coge la foto, le cambia el nombre para que no haya duplicados (le pone números delante) y la guarda en la carpeta de imágenes del servidor.
4. Guardado en BBDD	El sistema guarda toda la información escrita, enlazándola con el nombre de la foto que acaba de subir.
5. Finalización	La página se recarga y la nueva artesanía aparece automáticamente disponible para que la compren los clientes.

8.3.2.3. *Cambio de estado de un envío*

Paso a paso	Descripción del proceso interno
¿Quién lo hace?	El Administrador.

Requisito previo	Un cliente debe haber realizado una compra previamente.
1. Inicio	El administrador entra a su panel de gestión y localiza un pedido pendiente.
2. Acción	Despliega el menú de esa fila y selecciona "Enviado". Luego pulsa "Guardar".
3. Actualización	El sistema busca el número exacto de ese pedido en la base de datos y sustituye la palabra "pendiente" por "enviado".
4. Finalización	La pantalla del administrador se actualiza. A partir de ese momento, cuando el cliente entre a su perfil, verá la etiqueta verde de envío en su paquete.

8.4. Extractos de código reseñables

Por ejemplo, podemos ilustrar el código para la conexión, necesario para que la aplicación funcione en todas sus instancias.

```
<?php
namespace App\Config; // Indicamos dónde vive este archivo para el Autoload,
y olvidarnos de usar require_once, include... etc

//Para que no haya conflicto y php acceda a las clases globales, hacemos.
use PDO;
use PDOException;

class Conexion {

//Tal y como aplicamos en el módulo de DWES, dentro de una clase
    private $host = "localhost";
    private $db_name = "artesanos_db";
    private $user = "root";
    private $password = "";

// 2. Creamos una función que podremos llamar desde los Controladores
    public function conectar() {

        //construimos el dsn para la conexion
        $dsn = "mysql:host=" . $this->host . ";dbname=" . $this->db_name .
";charset=utf8";
```

```
/*Intentamos conectar a la BBDD con un try/catch para evitar que se cuelgue
si no hay conexión.
Recordar que PDO no devuelve errores (se queda callado)*/

try {

$conexion=new PDO($dsn,$this->user,$this->password);

//Configuramos los errores de PDO para poder manejar errores
$conexion->setAttribute(PDO::ATTR_ERRMODE,PDO::ERRMODE_EXCEPTION);

//Si funciona, devolvemos la conexion
return $conexion;

}
//Si hay errores, lo capturamos en una excepción que encerramos en una
variable $e
catch (PDOException $e){

    //Si no se conecta, la conexión muere, pero vemos un aviso
    die("Error al conectarse a la base de datos".$e->getMessage());
}

}

}

?>
```